6

223611496

8

223511323

5

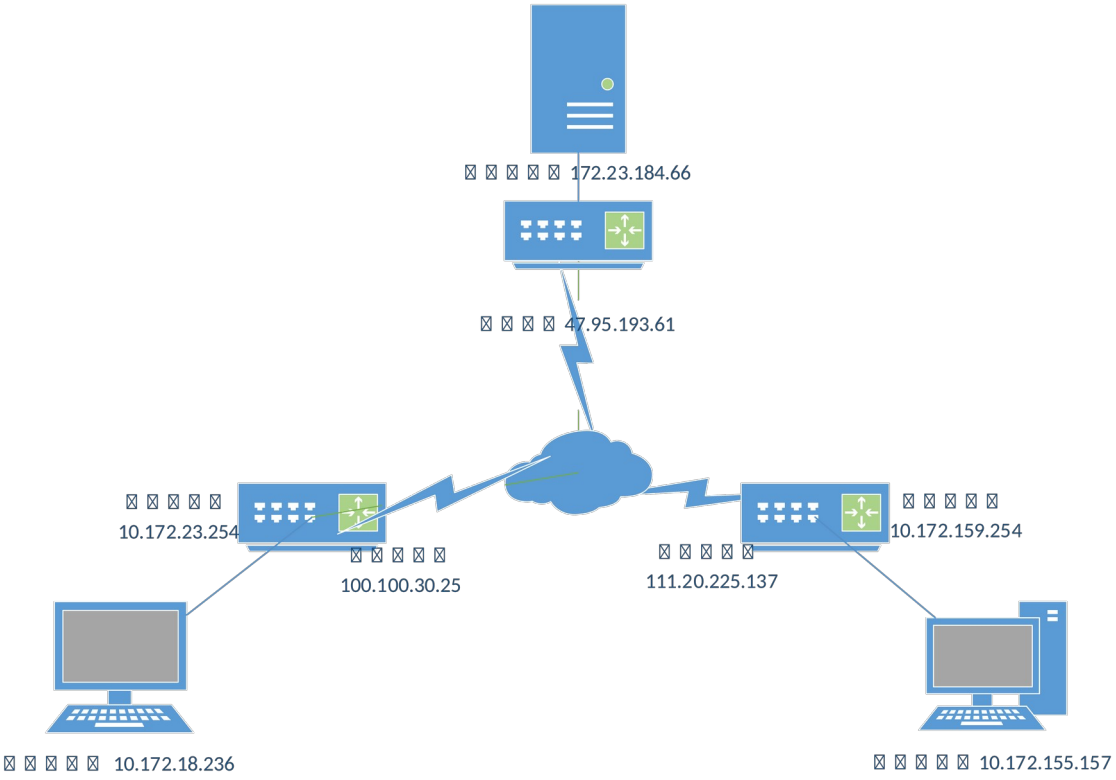
6 TCP

6.1

1. TCP TCP TCP TCP
2. TCP TCP
3. TCP TCP
4. TCP TCP
5. TCP

6.2

1. TCP TCP
2. TCP TCP TCP TCP MSS SACK Window Scale Timestamp
3. RTT MSS
4. TCP
5. TCP



6.3

6.3.1

2

6.3.2 TCP

1 PC2 web Apache IIS Python WEB 5MB

	172.23.184.66
	47.95.193.61
	pic.zip
	3.98 MB

2 PC1 Linux tcpdump

3 PC1 实验 **2** 编写的 **HTTP** WEB

3.1 实验 **2** 编写的 **HTTP** ms WEB 5MB

```
HTTP

    : 47.95.193.61
  : /pic.zip
  : 80
       IPv4    : 47.95.193.61
DNS     : 5.659  ms
TCP          : 25.624  ms
        : 1439.249  ms
HTTP      : HTTP/1.0  200  OK
HTTP      : 200
        : 4170186
         : 4169989
        : pic.zip

=====       HTTP      =====
GET /pic.zip HTTP/1.1
Host: 47.95.193.61
User-Agent: Huang YanShan
Accept: text/html,*/*
Accept-Encoding: identity
Connection: close


=====       =====
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.10.12
Date: Fri, 17 Apr 2026 02:24:51 GMT
Content-type: application/zip
Content-Length: 4169989
Last-Modified: Fri, 17 Apr 2026 02:12:50 GMT
```

3.2 HTTP □□□□

```
HTTP

    : 47.95.193.61
  : /pic.pcap
  : 80
       IPv4    : 47.95.193.61
DNS     : 2.064  ms
TCP          : 27.595  ms
        : 2186.886  ms
HTTP      : HTTP/1.0  200  OK
HTTP      : 200
        : 4475299
         : 4475089
        : pic.pcap

=====       HTTP      =====
GET /pic.pcap HTTP/1.1
Host: 47.95.193.61
User-Agent: Huang YanShan
Accept: text/html,*/*
Accept-Encoding: identity
Connection: close


=====       =====
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.10.12
Date: Fri, 17 Apr 2026 02:26:58 GMT
Content-type: application/vnd.tcpdump.pcap
Content-Length: 4475089
Last-Modified: Fri, 17 Apr 2026 02:24:56 GMT
```

4 TCP MSS SACK Window Scale Timestamp

4.1 MSS SACK Window Scale TCP

TCP	
MSS	TCP IP
SACK	
Window Scale	TCP header 16

4.2 PC1 TCP MSS SACK Window Scale

TCP		
MSS	1410	1460
SACK	Permitted	Permitted
Window Scale	7	7
	128	128

4.3 SACK

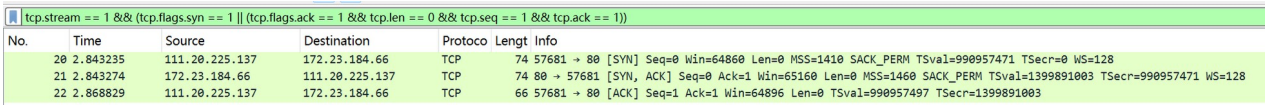
NO.670–NO.691 NO.670 seq=3156454633 NO.671 seq=3156456031 NO.672 seq 3156458827 NO.673 seq=3156457429 NO.672 NO.673 NO.690 ack=3156457429 NO.672 seq=3156458827 3156457429–3156458827 NO.691 ACK ack 3156457429 SACK [3156458827, 3156460225] 1398 NO.673 NO.692 ack 3156460225 SACK

NO.694 SACK [3156463021, 3156464419] NO.696 SACK [3156465817, 3156467215] SACK ack

NO.758–NO.760 SACK NO.758 ack=3156525931 SACK [3156527329, 3156528727] 3156525931–3156527329 NO.759 ack SACK [3156530125, 3156531523] [3156527329, 3156528727] NO.760 ack 3156528727 SACK [3156530125, 3156531523] SACK

5 TCP TCP RTT

5.1 □□ TCP □□□+□□□□□□□□□□□□□□□□□ **3** □ **TCP** □□

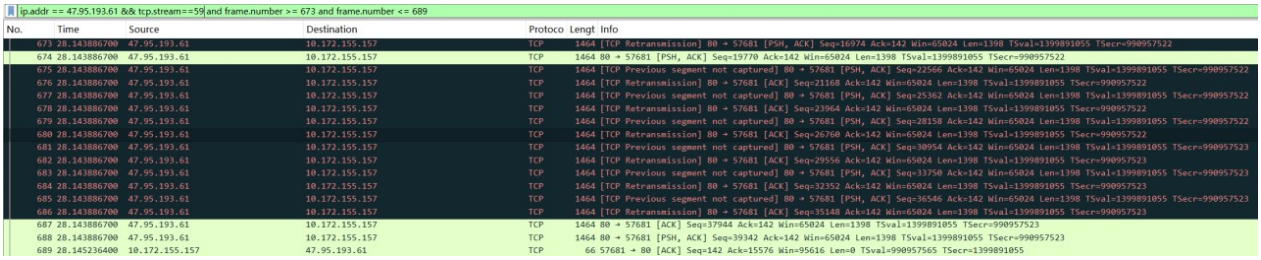


5.2 TCP RTT ms

RTT = t(22) - t(21)
= 2.868829 s - 2.843274 s
= 0.025555 s
= 25.555 ms

6 5MB **TCP** □ □□□□□□□□□□ TCP

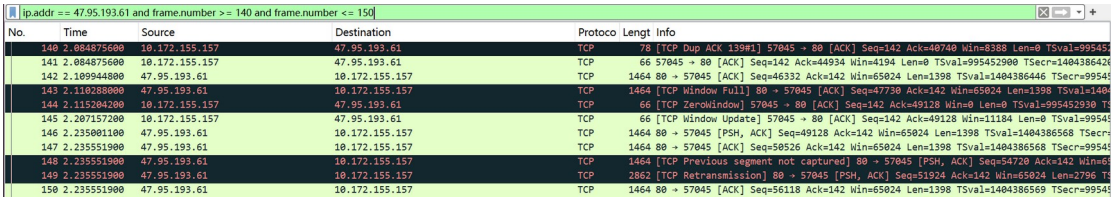
6.1 □□ TCP 10-15 □□□□ 1 □□ □+□□□□□□□□□□□□□□□□□□ **TCP** □□□



NO.672 NO.673 NO.691 ACK 3156457429 “ ” NO.673 NO.692 3156460225 TCP NO.672 NO.693 NO.674 RTO

6.2 □□□□ □□□□ □□□□ 10-15

No.	Seq Number	Ack Number			( )
670	3156454633	1678115225	508	1398	689
671	3156456031	1678115225	508	1398	690
672	3156458827	1678115225	508	1398	692
673	3156457429	1678115225	508	1398	692
674	3156460225	1678115225	508	1398	693
689	1678115225	3156456031	747	0	670



690	1678115225	3156457429	769	0	671
691	1678115225	3156457429	769	0	671
692	1678115225	3156460225	753	0	672 673
693	1678115225	3156461623	743	0	674

6.3 相关 TCP 9-13 流量控制 1 过滤器＋分组列表，列表中只给出填表中需要的 **TCP** 分组）

6.4 流量控制

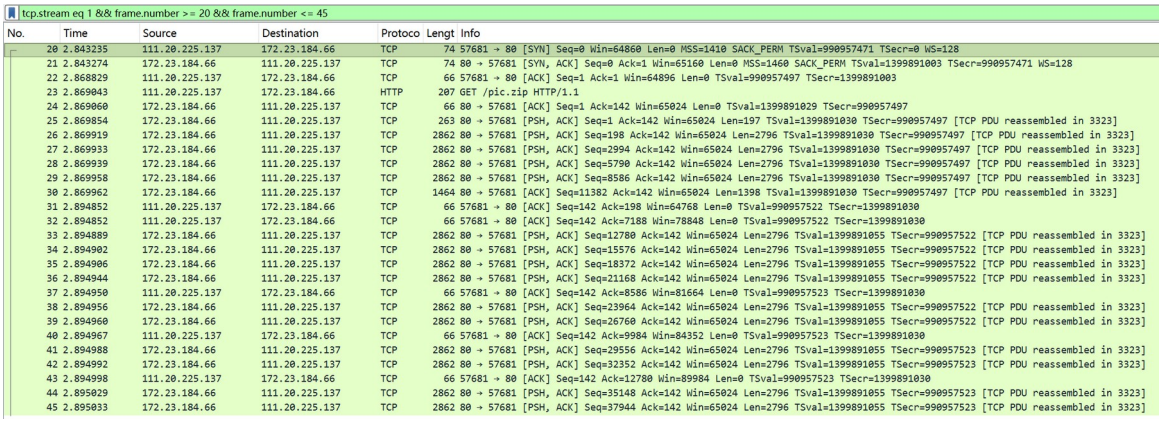
No.	Seq Number	Ack Number			( )
140	142	40740	8388	0	133
141	142	44934	4194	0	134
142	46332	142	508	1398	89
143	47730	142	508	1398	89
144	142	49128	0	0	143
145	142	49128	11184	0	143
146	49128	142	508	1398	89
147	50526	142	508	1398	89
148	54720	142	508	1398	89

6.5 6.3

ACK 140 8388
141 4194 144 0 Zero
Window 145 11184 146 147
148

7 TCP
cwnd

7.1 TCP 慢启动 1 分组列表，列表中只给出 的 **TCP** 分组）



tcp.stream eq 1 && frame.number >= 20 && frame.number <= 45

No.	Time	Source	Destination	Protoco	Lengt	Info
20	2.843235	111.20.225.137	172.23.184.66	TCP	74	57681 → 80 [SYN] Seq=0 Win=64860 Len=0 MSS=1410 SACK_PERM TSval=990957471 TSecr=0 WS=128
21	2.843274	172.23.184.66	111.20.225.137	TCP	74	80 → 57681 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=1399891003 TSecr=990957471 WS=128
22	2.868829	111.20.225.137	172.23.184.66	TCP	66	57681 → 80 [ACK] Seq=1 Ack=1 Win=64896 Len=0 TSval=990957497 TSecr=1399891003
23	2.869043	111.20.225.137	172.23.184.66	HTTP	207	GET /pic.zip HTTP/1.1
24	2.869060	172.23.184.66	111.20.225.137	TCP	66	80 → 57681 [ACK] Seq=1 Ack=142 Win=65024 Len=0 TSval=1399891029 TSecr=990957497
25	2.869854	172.23.184.66	111.20.225.137	TCP	263	80 → 57681 [PSH, ACK] Seq=1 Ack=142 Win=65024 Len=197 TSval=1399891030 TSecr=990957497 [TCP PDU reassembled in 3323]
26	2.869919	172.23.184.66	111.20.225.137	TCP	2862	80 → 57681 [PSH, ACK] Seq=198 Ack=142 Win=65024 Len=2796 TSval=1399891030 TSecr=990957497 [TCP PDU reassembled in 3323]
27	2.869933	172.23.184.66	111.20.225.137	TCP	2862	80 → 57681 [PSH, ACK] Seq=2994 Ack=142 Win=65024 Len=2796 TSval=1399891030 TSecr=990957497 [TCP PDU reassembled in 3323]
28	2.869939	172.23.184.66	111.20.225.137	TCP	2862	80 → 57681 [PSH, ACK] Seq=5790 Ack=142 Win=65024 Len=2796 TSval=1399891030 TSecr=990957497 [TCP PDU reassembled in 3323]
29	2.869958	172.23.184.66	111.20.225.137	TCP	2862	80 → 57681 [PSH, ACK] Seq=8586 Ack=142 Win=65024 Len=2796 TSval=1399891030 TSecr=990957497 [TCP PDU reassembled in 3323]
30	2.869962	172.23.184.66	111.20.225.137	TCP	1464	80 → 57681 [ACK] Seq=11382 Ack=142 Win=65024 Len=1398 TSval=1399891030 TSecr=990957497 [TCP PDU reassembled in 3323]
31	2.894852	111.20.225.137	172.23.184.66	TCP	66	57681 → 80 [ACK] Seq=142 Ack=198 Win=64768 Len=0 TSval=990957522 TSecr=1399891030
32	2.894852	111.20.225.137	172.23.184.66	TCP	66	57681 → 80 [ACK] Seq=142 Ack=7188 Win=78848 Len=0 TSval=990957522 TSecr=1399891030
33	2.894889	172.23.184.66	111.20.225.137	TCP	2862	80 → 57681 [PSH, ACK] Seq=12780 Ack=142 Win=65024 Len=2796 TSval=1399891055 TSecr=990957522 [TCP PDU reassembled in 3323]
34	2.894902	172.23.184.66	111.20.225.137	TCP	2862	80 → 57681 [PSH, ACK] Seq=15576 Ack=142 Win=65024 Len=2796 TSval=1399891055 TSecr=990957522 [TCP PDU reassembled in 3323]
35	2.894906	172.23.184.66	111.20.225.137	TCP	2862	80 → 57681 [PSH, ACK] Seq=18372 Ack=142 Win=65024 Len=2796 TSval=1399891055 TSecr=990957522 [TCP PDU reassembled in 3323]
36	2.894944	172.23.184.66	111.20.225.137	TCP	2862	80 → 57681 [PSH, ACK] Seq=21168 Ack=142 Win=65024 Len=2796 TSval=1399891055 TSecr=990957522 [TCP PDU reassembled in 3323]
37	2.894950	111.20.225.137	172.23.184.66	TCP	66	57681 → 80 [ACK] Seq=142 Ack=8586 Win=81664 Len=0 TSval=990957523 TSecr=1399891030
38	2.894956	172.23.184.66	111.20.225.137	TCP	2862	80 → 57681 [PSH, ACK] Seq=23964 Ack=142 Win=65024 Len=2796 TSval=1399891055 TSecr=990957522 [TCP PDU reassembled in 3323]
39	2.894960	172.23.184.66	111.20.225.137	TCP	2862	80 → 57681 [PSH, ACK] Seq=26760 Ack=142 Win=65024 Len=2796 TSval=1399891055 TSecr=990957522 [TCP PDU reassembled in 3323]
40	2.894967	111.20.225.137	172.23.184.66	TCP	66	57681 → 80 [ACK] Seq=142 Ack=9984 Win=84352 Len=0 TSval=990957523 TSecr=1399891030
41	2.894988	172.23.184.66	111.20.225.137	TCP	2862	80 → 57681 [PSH, ACK] Seq=29556 Ack=142 Win=65024 Len=2796 TSval=1399891055 TSecr=990957523 [TCP PDU reassembled in 3323]
42	2.894992	172.23.184.66	111.20.225.137	TCP	2862	80 → 57681 [PSH, ACK] Seq=32352 Ack=142 Win=65024 Len=2796 TSval=1399891055 TSecr=990957523 [TCP PDU reassembled in 3323]
43	2.894998	111.20.225.137	172.23.184.66	TCP	66	57681 → 80 [ACK] Seq=142 Ack=12780 Win=89984 Len=0 TSval=990957523 TSecr=1399891030
44	2.895029	172.23.184.66	111.20.225.137	TCP	2862	80 → 57681 [PSH, ACK] Seq=35148 Ack=142 Win=65024 Len=2796 TSval=1399891055 TSecr=990957523 [TCP PDU reassembled in 3323]
45	2.895033	172.23.184.66	111.20.225.137	TCP	2862	80 → 57681 [PSH, ACK] Seq=37944 Ack=142 Win=65024 Len=2796 TSval=1399891055 TSecr=990957523 [TCP PDU reassembled in 3323]

7.2 □□□

1. 22 t=2.868829s HTTP 23
2. 25-30 ACK 6

   25: seq=1, len=197 HTTP

   26: seq=198, len=2796 = 2×1398 TSO 2 MSS

   27: seq=2994, len=2796

   28: seq=5790, len=2796

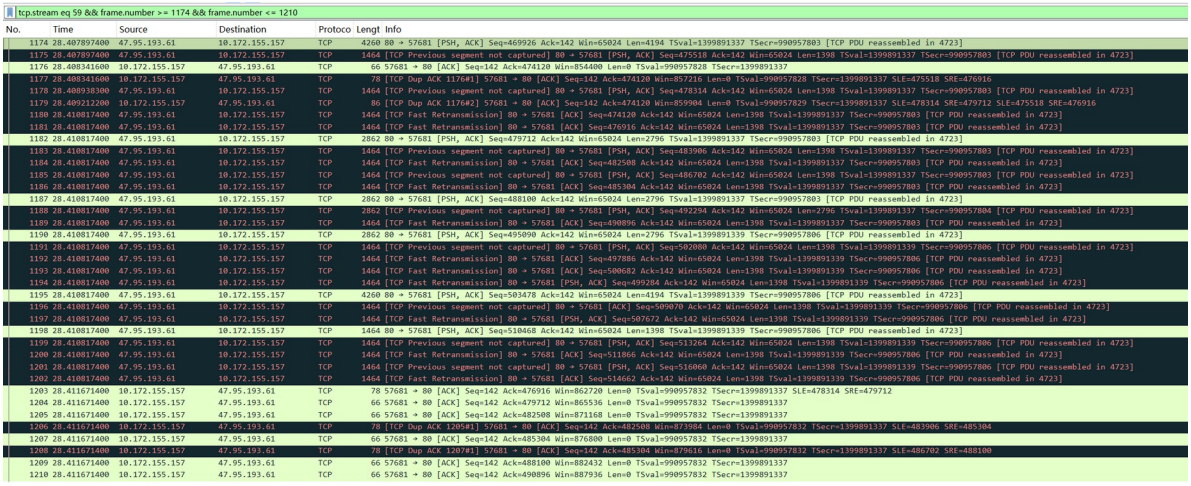
   29: seq=8586, len=2796

   30: seq=11382, len=1398

   197+2796×4+1398 = 12779 = 10 MSS cwnd
3. ACK 31-32 t=2.894852s 25ms ≈1 RTT

   ACK cwnd 1 MSS
4. 33-45 cwnd 20 MSS 27960

7.3 TCP □□□□ 1 □□□□□□□□□□□ □ **TCP** □□□



7.4 TCP □□□□

1.

1174 seq=469926, len=4194 469926-474119

1175 seq=475518, len=1398 475518-476915

474120-475517 1 MSS

2. ACK

1176 ACK ack=474120 474120

1177 ACK ack=474120 [TCP Dup ACK #1] 474120

1179 ACK ack=474120 [TCP Dup ACK #2] 474120

3.

1180 seq=474120, len=1398 [TCP Fast Retransmission]

474120-475517

1181-1202

4.

1203 ACK ack=476916, 479712 ...

7.5 cwnd MSS

TCP cwnd MSS	10

6.3.3 TCP

8 TCP

TCP

```
import argparse
import os
import socket
from pathlib import Path

BUFFER_SIZE = 4096
DEFAULT_HOST = "0.0.0.0"
DEFAULT_PORT = 9000

def recv_line(conn: socket.socket) -> str:
    data = bytearray()
    while True:
        chunk = conn.recv(1)
        if not chunk:
            break
        if chunk == b"\n":
            break
        data.extend(chunk)
    return data.decode("utf-8", errors="replace").strip()

def safe_resolve(base_dir: Path, requested_name: str) -> Path | None:
    if not requested_name:
        return None

    candidate = (base_dir / requested_name).resolve()
    try:
        candidate.relative_to(base_dir)
    except ValueError:
        return None

    return candidate

def send_file(conn: socket.socket, file_path: Path) -> None:
    file_size = file_path.stat().st_size
    header = f"OK {file_size}\n".encode("utf-8")
    conn.sendall(header)
    print(f"           : {header.decode('utf-8').strip()}")
```

```
    with file_path.open("rb") as file_obj:
        while True:
            chunk = file_obj.read(BUFFER_SIZE)
            if not chunk:
                break
            conn.sendall(chunk)

def handle_client(conn: socket.socket, addr: tuple[str, int], base_dir: Path) -> None:
    print(f"           : {addr[0]}:{addr[1]}")

    try:
        requested_name = recv_line(conn)
        print(f"           : {requested_name!r}")

        file_path = safe_resolve(base_dir, requested_name)
        if file_path is None:
            conn.sendall("ERR               \n".encode("utf-8"))
            print("       :               ")
            return

        if not file_path.is_file():
            conn.sendall("ERR                \n".encode("utf-8"))
            print("       :           ")
            return

        send_file(conn, file_path)
        print(f"       : {file_path.name} ({file_path.stat().st_size} bytes)")
    except OSError as exc:
        print(f"                  : {exc}")
    finally:
        conn.close()
        print(f"         : {addr[0]}:{addr[1]}")

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="TCP               ")
    parser.add_argument("--host", default=DEFAULT_HOST, help="               0.0.0.0")
    parser.add_argument("--port", type=int, default=DEFAULT_PORT, help="               9000")
    parser.add_argument(
```

```
        "--dir",
        default=".",
        help="                                ",
    )
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    base_dir = Path(args.dir).resolve()

    if not base_dir.is_dir():
        raise SystemExit(f"             : {base_dir}")

    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((args.host, args.port))
    server.listen(5)

    print("=" * 60)
    print("TCP                    ")
    print(f"       : {args.host}")
    print(f"       : {args.port}")
    print(f"       : {base_dir}")
    print("=" * 60)

    try:
        while True:
            conn, addr = server.accept()
            handle_client(conn, addr, base_dir)
    except KeyboardInterrupt:
        print("\n            ")
    finally:
        server.close()

if __name__ == "__main__":
    main()
```

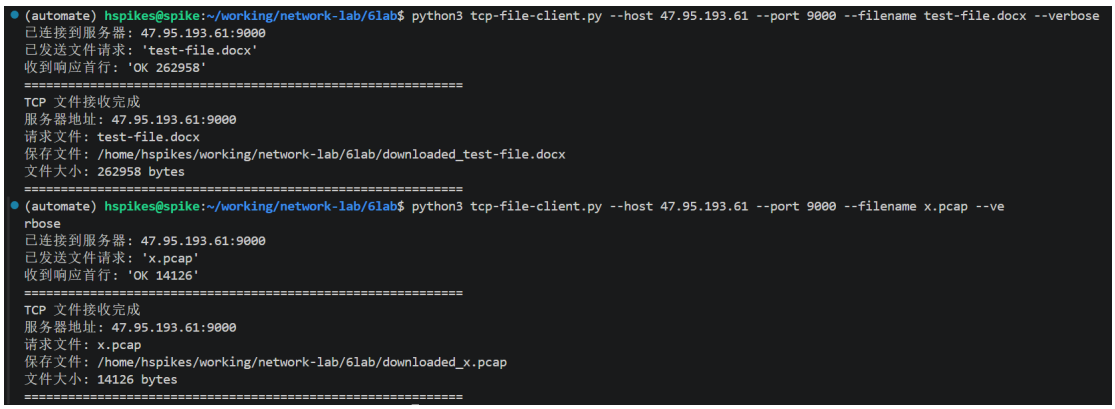
9 8

```
import argparse
import socket
from pathlib import Path

BUFFER_SIZE = 4096
DEFAULT_HOST = "127.0.0.1"
DEFAULT_PORT = 9000
DEFAULT_TIMEOUT = 10

def recv_line(sock: socket.socket) -> str:
    data = bytearray()
    while True:
        chunk = sock.recv(1)
        if not chunk:
            break
        if chunk == b"\n":
            break
        data.extend(chunk)
    return data.decode("utf-8", errors="replace").strip()

def request_filename(args: argparse.Namespace) -> str:
    if args.filename:
        return args.filename
    return input("                : ").strip()

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="TCP             ")
    parser.add_argument("--host", default=DEFAULT_HOST, help="             127.0.0.1")
    parser.add_argument("--port", type=int, default=DEFAULT_PORT, help="             9000")
    parser.add_argument("--filename", help="                          ")
    parser.add_argument(
        "--output",
        help="                      downloaded_<      >",
    )
```

```
    parser.add_argument(
        "--timeout",
        type=float,
        default=DEFAULT_TIMEOUT,
        help="                    10    ",
    )
    parser.add_argument("--verbose", action="store_true", help="                    ")
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    requested_name = request_filename(args)
    if not requested_name:
        raise SystemExit("            ")

    output_name = args.output or f"downloaded_{Path(requested_name).name}"

    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.settimeout(args.timeout)

    try:
        client.connect((args.host, args.port))
        if args.verbose:
            print(f"            : {args.host}:{args.port}")

        client.sendall(f"{requested_name}\n".encode("utf-8"))
        if args.verbose:
            print(f"            : {requested_name!r}")

        response_line = recv_line(client)
        if args.verbose:
            print(f"          : {response_line!r}")
        if not response_line:
            raise SystemExit("                  ")

        if response_line.startswith("ERR "):
            raise SystemExit(f"            : {response_line[4:]}")
```

```
        if not response_line.startswith("OK "):
            raise SystemExit(f"        : {response_line}")

        file_size = int(response_line.split(maxsplit=1)[1])
        received_size = 0
        output_path = Path(output_name)

        with output_path.open("wb") as file_obj:
            while received_size < file_size:
                chunk = client.recv(min(BUFFER_SIZE, file_size - received_size))
                if not chunk:
                    raise SystemExit("                          ")
                file_obj.write(chunk)
                received_size += len(chunk)

        print("=" * 60)
        print("TCP             ")
        print(f"          : {args.host}:{args.port}")
        print(f"       : {requested_name}")
        print(f"       : {output_path.resolve()}")
        print(f"       : {received_size} bytes")
        print("=" * 60)
    except socket.timeout as exc:
        raise SystemExit(
            "                                                    HTTP            "
        ) from exc
    finally:
        client.close()

if __name__ == "__main__":
    main()
```

10 500KB



11 TCP

```
import argparse
import socket
from pathlib import Path

BUFFER_SIZE = 4096
DEFAULT_HOST = "0.0.0.0"
DEFAULT_PORT = 9000

def recv_line(conn: socket.socket) -> str:
    data = bytearray()
    while True:
        chunk = conn.recv(1)
        if not chunk:
            break
        if chunk == b"\n":
            break
        data.extend(chunk)
    return data.decode("utf-8", errors="replace").strip()

def safe_resolve(base_dir: Path, requested_name: str) -> Path | None:
    if not requested_name:
        return None

    candidate = (base_dir / requested_name).resolve()
    try:
```

```
            candidate.relative_to(base_dir)
        except ValueError:
            return None

        return candidate

def send_file(conn: socket.socket, file_path: Path) -> None:
        file_size = file_path.stat().st_size
        header = f"OK {file_size}\n".encode("utf-8")
        conn.sendall(header)
        print(f"           : {header.decode('utf-8').strip()}")

        with file_path.open("rb") as file_obj:
            while True:
                chunk = file_obj.read(BUFFER_SIZE)
                if not chunk:
                    break
                conn.sendall(chunk)

def handle_client(conn: socket.socket, addr: tuple[str, int], base_dir: Path) -> None:
        client_ip, client_port = addr
        server_ip, server_port = conn.getsockname()

        print(f"             : {client_ip}:{client_port}")
        print(f"               : {server_ip}:{server_port}")

        try:
            requested_name = recv_line(conn)
            print(f"           : {requested_name!r}")

            file_path = safe_resolve(base_dir, requested_name)
            if file_path is None:
                conn.sendall("ERR               \n".encode("utf-8"))
                print("       :               ")
                return

            if not file_path.is_file():
                conn.sendall("ERR                \n".encode("utf-8"))
```

```
                print("       :           ")
                return

            send_file(conn, file_path)
            print(f"       : {file_path.name} ({file_path.stat().st_size} bytes)")
        except OSError as exc:
            print(f"                  : {exc}")
        finally:
            conn.close()
            print(f"         : {client_ip}:{client_port}")

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="TCP                ")
    parser.add_argument("--host", default=DEFAULT_HOST, help="              0.0.0.0")
    parser.add_argument("--port", type=int, default=DEFAULT_PORT, help="              9000")
    parser.add_argument(
        "--dir",
        default=".",
        help="                                ",
    )
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    base_dir = Path(args.dir).resolve()

    if not base_dir.is_dir():
        raise SystemExit(f"             : {base_dir}")

    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((args.host, args.port))
    server.listen(5)

    print("=" * 60)
    print("TCP                    ")
    print(f"       : {args.host}")
    print(f"       : {args.port}")
```

```
    print(f"        : {base_dir}")
    print("=" * 60)

    try:
        while True:
            conn, addr = server.accept()
            handle_client(conn, addr, base_dir)
    except KeyboardInterrupt:
        print("\n            ")
    finally:
        server.close()

if __name__ == "__main__":
    main()
```

6.4

12

12.1 TCP

TCP 发送方 ACK cwnd ssthresh

12.2 TCP

TCP 接收方 ACK rwnd

12.3 TCP

TCP 面向连接、可靠、按字节流传输 TCP TCP

TCP TCP